

BTAF Protocol (BTAF)

SMART CONTRACT

Security Audit

Performed on Contract:

InvoiceHandler.sol

476570B5E066585EC3542705EC92A949

(90 days, 65% APY) Staking.sol

0xBa2fa86dAC6958492F04ea453937E61E56A9BfD3

(90 days, 35% APY) Staking.sol

0xb88A8F8BFBBBf5ba911CDe89C8697f5dc22DD27F

(2 years) Staking.sol

0x3878bfd5ef558BDfD7b3709E9119c1e1De91227f

(3 years) Staking.sol

0x87A0939B9048921f2ee4f82B90fC31042d29ec12

(1 day - testing) Staking.sol

04412b09051359c9093a0dfd2064c543

Platform

BSC

hashlock.com.au

FEBRUARY 2023

Table of Contents

Executive Summary	4
Project Context	4
Roadmap	7
Audit scope	8
Security Rating	9
Standardised Checks	10
Intended Smart Contract Functions	12
Function List	15
Function Analysis	18
Code Quality	21
Audit Resources	21
Dependencies	21
Severity Definitions	22
Audit Findings	22
Centralisation	31
Owner executable functions	31
Conclusion	32
Our Methodology	33
Disclaimers	35
About Hashlock	36
Appendix	37

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

Bitcoin Trend and Forecast's (BTAF) team partnered with Hashlock to conduct a security audit of their BNB Smart Chain (BSC) Invoice Handler and Staking smart contracts after Hashlocks previous audit of their token. Hashlock manually and proactively reviewed the code in order to ensure the project was safe for launch, continued use and to give confidence to their community.

Project Context

[BitcoinTAF's](#) team previously launched [BTAF token](#) before the token was audited by Hashlock. This subsequent audit focuses on their staking and invoice handler contracts. BitcoinTAF is a leading platform for cryptocurrency trading education, technical data, and fundamental research. See the whitepaper [here](#) for more context.

Token Name: BTAF Token (BTAF)

Contract Addresses (testnet):

InvoiceHandler: 0xF7AF6DA5fCBcAABCf411bF6A3D5e31DF21c78Ea8

Staking: 0xBD08837459F554A9705547cE5E9b1Ad9137cE26a

Compiler Version: v0.8.3+commit.8d00100c

Logo:

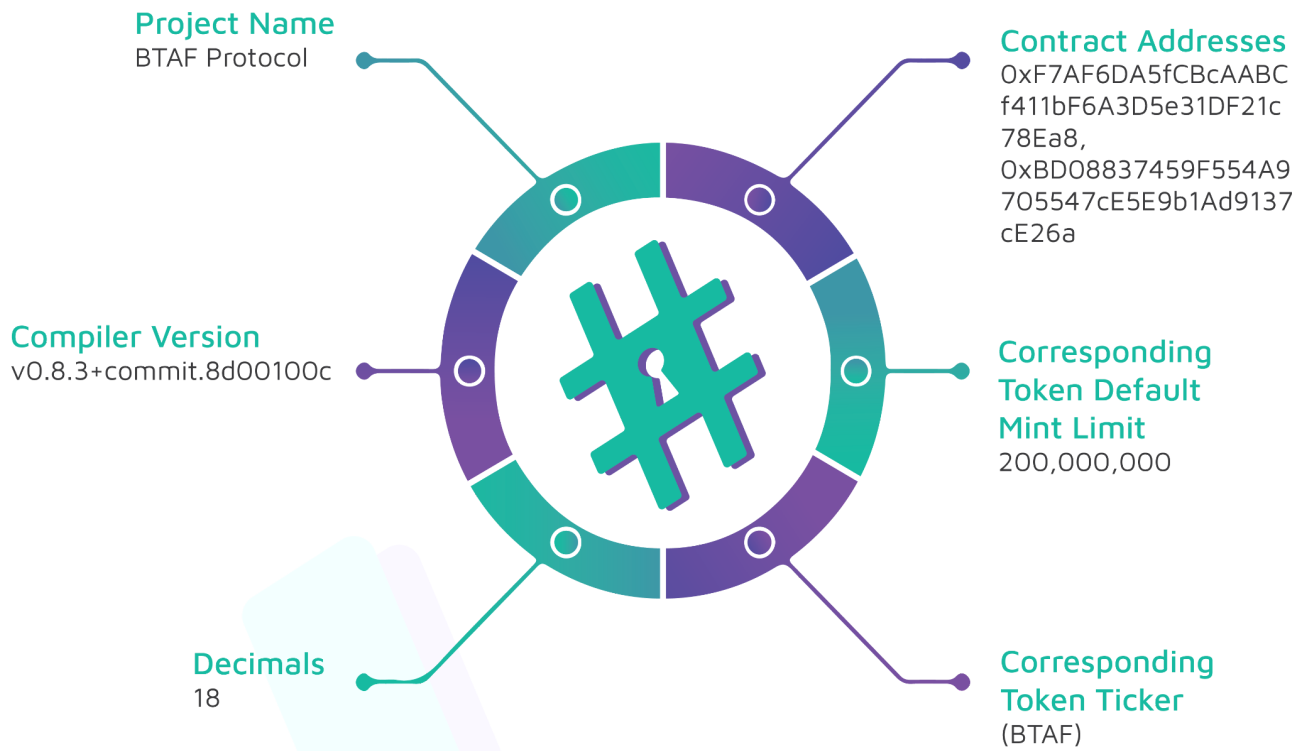


Mascot: (Tuffy)



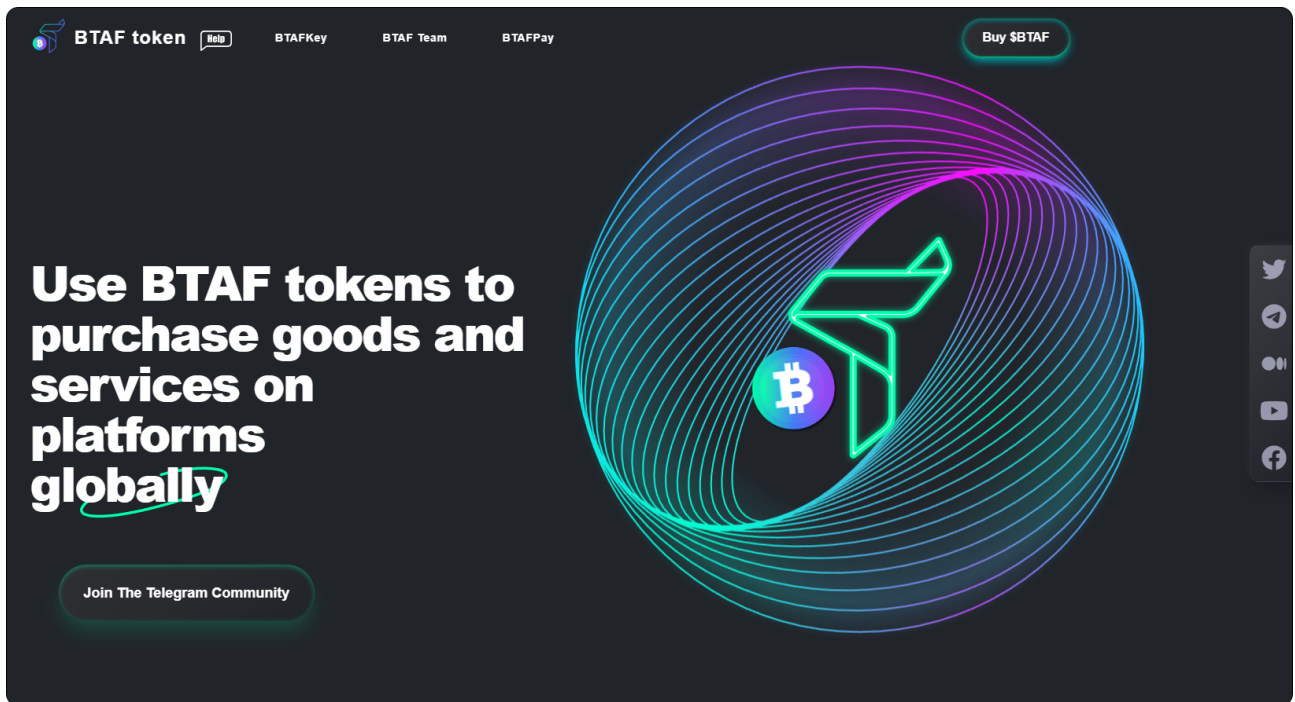
#Hashlock.

Hashlock Pty Ltd

Visualised Context:

Platform Visuals:

BTAF token website:



BitcoinTAF website:



Roadmap

Launch:

- #BTAF token
- Deploy Governance
- Deploy on Pancake Swap

Q4 2022:

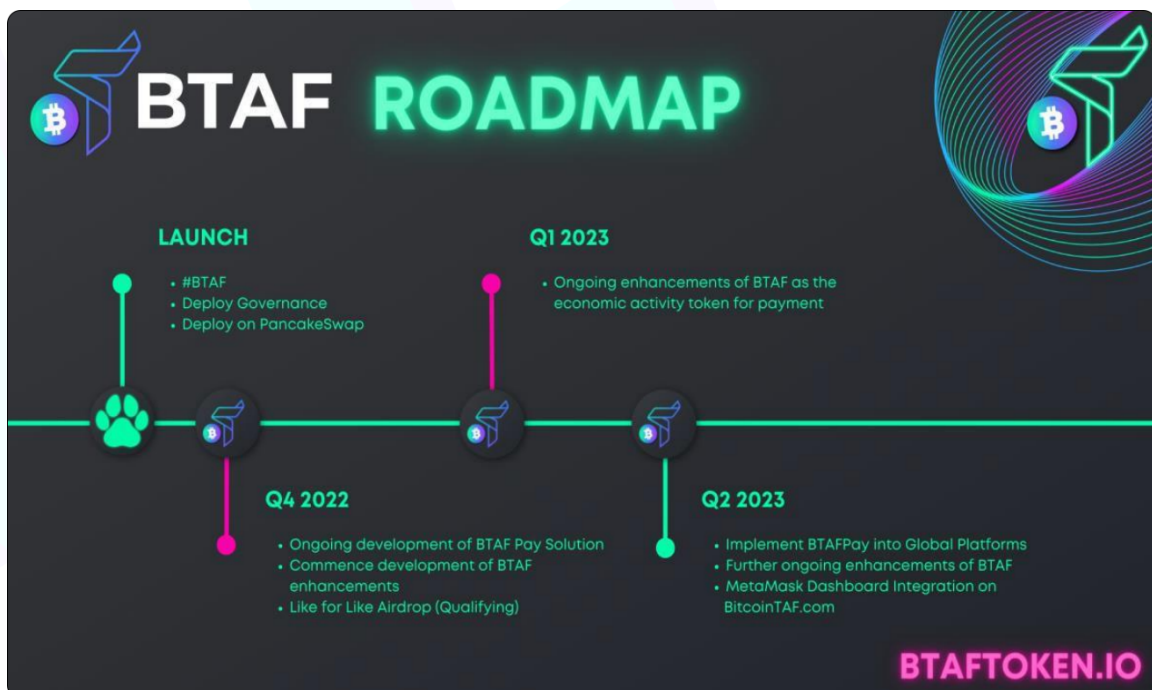
- Ongoing development of BTAF Pay Solution
- Commence development of BTAF enhancements
- Like for Like Airdrop (Qualifying)

Q1 2023:

- Ongoing enhancements of BTAF as the economic activity token for payment

Q2 2023:

- Implement BTAFPay into Global Platforms, further ongoing enhancements of BTAF Metamask dashboard integration on BitcoinTAF.com



Audit scope

Description	Project review and security analysis report for BTAF Protocol smart contracts and other factors.
Platform	Ethereum / Solidity
Audit Date	Feb 2023
Contract 1	InvoiceHandler.sol
Contract 1 MD5 Hash	476570B5E066585EC3542705EC92A949
Contract 2	(90 days, 65% APY) Staking.sol
Contract 2 MD5 Hash	04412b09051359c9093a0dfd2064c543
Revised Contract 2	0xBa2fa86dAC6958492F04ea453937E61E56A9BfD3
Contract 3	(90 days, 35% APY) Staking.sol
Contract 3 MD5 Hash	00be17e75f8bf3ac72370207f287b766
Revised Contract 3	0xb88A8F8BFBBBf5ba911CDe89C8697f5dc22DD27F
Contract 4	(2 years) Staking.sol
Contract 4 MD5 Hash	04412b09051359c9093a0dfd2064c543
Revised Contract 4	0x3878bfd5ef558BDfD7b3709E9119c1e1De91227f
Contract 5	(3 years) Staking.sol
Contract 5 MD5 Hash	04412b09051359c9093a0dfd2064c543
Revised Contract 5	0x87A0939B9048921f2ee4f82B90fC31042d29ec12
Contract 6	(1 day - testing) Staking.sol
Contract 6 MD5 Hash	04412b09051359c9093a0dfd2064c543

Security Rating

After our audit and analysis, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We have identified some medium severity vulnerabilities that will need to be addressed before mainnet launch.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for fully decentralised, highly secure projects with little owner influence.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit overview section. General overview is presented in the Function list section and all identified issues can be found in the Audit overview section.

Hashlock found:

0 Critical severity vulnerabilities

0 High severity vulnerabilities

2 Medium severity vulnerabilities

5 Low severity vulnerabilities

3 Very low, informational, best practice severity vulnerabilities

Caution: Hashlock's audit's do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Standardised Checks

Main Category	Subcategory	Result
General Code Checks	Solidity/compiler version stated	Passed
	Consistent pragma version across each contract	Passed
	Outdated Solidity Version	Reviewed
	Overflow/underflow	Passed
	Correct checks, effects, interaction order	Reviewed
	Lack of check on input parameters	Reviewed
	Function input parameters check bypass	Passed
	Correct Access control	Reviewed
	Built in emergency features	Passed
	Correct event logs	Reviewed
	Human/contract checks bypass	Passed
	Random number generation/use vulnerability	Passed
	Fallback function misuse	Passed
	Race condition	Passed
	Logical vulnerability	Passed
	Features claimed	Passed
	delegatecall() vulnerabilities	Passed
	Other programming issues	Reviewed
Code Specification	Correctly declared function visibility	Passed
	Correctly declared variable storage location	Passed
	Use keywords/functions to be deprecated	Passed
	Unused code	Passed
Gas Optimization	"Out of Gas" Issue	Passed
	High consumption 'for/while' loop	Passed

	High consumption 'storage' storage	Passed
	Assert() misuse	Passed
Tokenomics Risk	The maximum limit for mintage not set	Passed
	"Short Address" Attack	Passed
	"Double Spend" Attack	Passed

Initial Audit Result: Failed

Revised Audit Result: Passed

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
File 1 Staking.sol • Maturation Days: 90 Days • Reward Rate: 16.027% • Starting Burn Rate: 20% • Open Zeppelin standard code is used. Features: • Deposit at any time. • Withdraw penalty decreases over time to 0 at maturation date. • Investors can create a new Deposit at any time. An investor must interact with each Deposit separately as if each were an individual contract. • Users can claim current rewards at any time. • Reward Rate: – A multiplier applied to the staking amount, used to calculate the reward tokens due to an investor at Maturation. • Transfer an amount of tokens into a new Deposit. The Deposit is given a Maturation date of now + Maturation Days.	Reviewed
File 2 Staking.sol • Maturation Days: 2 years • Reward Rate: 50% • Starting Burn Rate: 80% • Open Zeppelin standard code is used. Features: • Deposit at any time. • Withdraw penalty decreases over time to 0 at	Reviewed

maturation date. • Investors can create a new Deposit at any time. An investor must interact with each Deposit separately as if each were an individual contract. • Users can claim current rewards at any time. • <u>Reward Rate:</u> – A multiplier applied to the staking amount, used to calculate the reward tokens due to an investor at Maturation. • Transfer an amount of tokens into a new Deposit. The Deposit is given a Maturation date of now + Maturation Days.	
File 3 Staking.sol • Maturation Days: 3 years • Reward Rate: 120% • Starting Burn Rate: 80% • Open Zeppelin standard code is used. <u>Features:</u> • Deposit at any time. • Withdraw penalty decreases over time to 0 at maturation date. • Investors can create a new Deposit at any time. An investor must interact with each Deposit separately as if each were an individual contract. • Users can claim current rewards at any time. • <u>Reward Rate:</u> – A multiplier applied to the staking amount, used to calculate the reward tokens due to an investor at Maturation. • Transfer an amount of tokens into a new Deposit. The Deposit is given a Maturation date of now + Maturation Days.	Reviewed
File 4 Staking.sol	Reviewed

• Maturation Days: 1 day • Reward Rate: 10% • Starting Burn Rate: 20% • Open Zeppelin standard code is used. <u>Features:</u> • Deposit at any time. • Withdraw penalty decreases over time to 0 at maturation date. • Investors can create a new Deposit at any time. An investor must interact with each Deposit separately as if each were an individual contract. • Users can claim current rewards at any time. • <u>Reward Rate:</u> – A multiplier applied to the staking amount, used to calculate the reward tokens due to an investor at Maturation. • Transfer an amount of tokens into a new Deposit. The Deposit is given a Maturation date of now + Maturation Days.	
File 2 InvoiceHandler.sol • Open Zeppelin standard code is used. <u>Ownership Control:</u> • Owner can change the registered invoice signer for the contract.	Contract achieves this functionality.

Function List

Staking.sol contracts

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	Refer Audit Findings
2	onlyOwner	modifier	Passed	Refer Audit Findings
3	owner	read	Passed	No Issue
4	_checkOwner	internal	Passed	No Issue
5	renounceOwnership	write	access only Owner	No Issue
6	transferOwnership	write	access only Owner	No Issue
7	_transferOwnership	internal	Passed	No Issue
8	getStake	read	Passed	No Issue
9	getInvestorStake	read	Passed	No Issue
10	getStakeCount	read	Passed	No Issue
11	getInvestorStakeCount	read	Passed	No Issue
12	getInvestorCount	read	Passed	No Issue
13	getInvestor	read	Passed	No Issue
14	depositStake	write	Critical operation lacks event log, Function input parameters lack of check	Refer Audit Findings
15	previewInvestorClaimReward	read	Passed	No Issue
16	claimReward	write	Passed	Refer Audit Findings
17	previewInvestorWithdrawStake	read	Passed	No Issue
18	withdrawStake	write	Passed	No Issue
19	refundInvestorStake	write	Passed	No Issue

20	getTimeRemaining	read	Passed	No Issue
----	------------------	------	--------	----------

InvoiceHandler.sol

Functions

	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	onlyOwner	modifier	Passed	No Issue
3	owner	read	Passed	No Issue
4	_checkOwner	internal	Passed	No Issue
5	renounceOwnership	write	access only Owner	No Issue
6	transferOwnership	write	access only Owner	No Issue
7	_transferOwnership	internal	Passed	No Issue
8	setInvoiceSigner	write	access only Owner	No Issue
9	isValidSignature	internal	Passed	No Issue
10	isValidInvoiceSignature	read	Passed	No Issue
11	payInvoice	write	Passed	No Issue
12	hashInvoice	write	Passed	No Issue
1	constructor	write	Passed	No Issue
2	onlyOwner	modifier	Passed	No Issue
3	owner	read	Passed	No Issue
4	_checkOwner	internal	Passed	No Issue
5	renounceOwnership	write	access only Owner	No Issue
6	transferOwnership	write	access only Owner	No Issue
7	_transferOwnership	internal	Passed	No Issue
8	setInvoiceSigner	write	access only Owner	No Issue
9	isValidSignature	internal	Passed	No Issue
10	isValidInvoiceSignature	read	Passed	No Issue

11	payInvoice	write	Passed	No Issue
12	hashInvoice	write	Passed	No Issue

Function Analysis

Hashlock has analysed and explained the overall function of the contracts below, to be accessible for a non-technical audience:

Staking contracts

depositStake

```

1  BtaTest.sol
2  * @notice Deposit a stake. The stake will be added to the caller's stakes array. It will include a maturation date which is the current date + maturationDays
3  * @param amount Amount to stake
4  */
5  function depositStake(uint256 amount) public {
6
7      // If the maturation date has passed, revert
8      require(maturationDate > block.timestamp, "Staking period has ended");
9
10     // Check that amount is not zero
11     require(amount > 0, "Amount must be greater than zero");
12
13     // Deduct the staking fee
14     uint256 fee = Math.mulDiv(amount, stakingFeeRate, PRECISION * 100, Math.Rounding.Zero);
15     uint256 netStakeAmount = amount - fee;
16
17     // Calculate reward
18     // Time since deployment. The reward rate decreases linearly as this value increases
19     uint256 timeToMaturation = maturationDate - block.timestamp;
20
21     uint256 maxReward = Math.mulDiv(netStakeAmount, rewardRate, PRECISION * 100, Math.Rounding.Zero);
22     uint256 reward = Math.mulDiv(maxReward, timeToMaturation, maturationPeriod, Math.Rounding.Zero);
23
24     Stake memory stake = Stake(netStakeAmount, reward, address(msg.sender), block.timestamp, 0, 0, 0, fee);
25
26     // Add the investor to the investors array if they are not already in it
27     if (stakes[address(msg.sender)].length == 0) {
28         investors.push(address(msg.sender));
29     }
30     // Add the new Stake to the stakes array
31     stakes[address(msg.sender)].push(stake);
32
33     // Transfer the amount from the staker to the contract
34     IERC20(token).transferFrom(address(msg.sender), address(this), netStakeAmount);
35
36     // Transfer the fee to the owner address
37     IERC20(token).transferFrom(address(msg.sender), owner(), fee);
38
39     // Emit the StakeDeposited event
40     emit StakeDeposited(address(msg.sender), stakes[address(msg.sender)].length - 1, netStakeAmount, fee);
41 }

```

Users can deposit the token, the contract records the data in an array, and adds the user to the investors array. There is a set fee for depositing Stakes.

claimReward

```

137     function claimReward(uint256 index) public {
138         // Get the stake
139         uint256 rewardAmount = previewInvestorClaimReward(address(msg.sender), index);
140
141         if (rewardAmount > 0) {
142             Stake storage stake = stakes[address(msg.sender)][index];
143             stake.totalRewardClaimed += rewardAmount;
144             IERC20(token).transfer(stake.staker, rewardAmount);
145             emit RewardClaimed(stake.staker, index, rewardAmount);
146         }
147     }
148

```

Users can claim the rewards they have earned, calculated based on if the maturation date has passed or not, and the staking terms, in the `previewInvestorClaimReward` public view function.

withdrawStake

```

169     function withdrawStake(uint256 index) public {
170         // Preview
171         (uint256 withdrawAmount, uint256 burnAmount) = previewInvestorWithdrawStake(address(msg.sender), index);
172         Stake storage stake = stakes[address(msg.sender)][index];
173
174         // Burn the burn amount by sending it to address 1
175         if (burnAmount > 0) {
176             stake.stakeBurnt = burnAmount;
177             IERC20(token).transfer(address(1), burnAmount);
178         }
179         stake.dateWithdrawn = block.timestamp;
180
181         // Send the remaining amount to the staker
182         IERC20(token).transfer(stake.staker, withdrawAmount);
183
184         emit StakeWithdrawn(stake.staker, index, withdrawAmount, burnAmount);
185     }
186 }

```

Users can withdraw their stake, cancelling their investment, burning a portion of their investment / rewards and withdrawing the remaining balance.

InvoiceHandler Contract

payInvoice

```
Staking.sol

196     function payInvoice(
197         uint256 invoiceId,
198         uint256 amount,
199         Signature calldata sig
200     ) public {
201         require(
202             isValidInvoiceSignature(invoiceId, amount, sig),
203             "Payment details do not match signature"
204         );
205         require(!invoicePayments[invoiceId].paid, "Invoice already paid");
206         require(amount > 0, "Amount must be greater than 0");
207         _token.safeTransferFrom(msg.sender, owner(), amount);
208         invoicePayments[invoiceId] = InvoicePayment(
209             invoiceId,
210             amount,
211             msg.sender,
212             true
213         );
214         emit InvoicePaid(invoiceId, msg.sender, amount);
215     }
```

Public can pay invoices tracked via an invoice ID, setting the boolean value to true, meaning paid.

Code Quality

This audit scope involves the solidity smart contracts of the BTAF project, as outlined in the Audit Scope section. All contracts, libraries and interfaces follow standard best practices and avoid unnecessary complexity that increases the likelihood of exploitation.

All code is well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the smart contract code in the form of testnet links.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are very helpful in understanding the overall architecture of the protocol.

We were also provided with project documentation and links to assist with our understanding of the project's architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
Critical	Critical vulnerabilities are of immediate concern, and allow hackers or malicious code to steal funds, take ownership of the contract, or other devastating actions.
High	High vulnerabilities are not attacked as easily, but still have significant implications on the protocol.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Informational	These are mostly preferred factors that won't affect security.

Audit Findings

Critical

No critical vulnerabilities were found.

High

- (1) Excess tokens are locked in contract.

Excess tokens earned by the staking contract or sent to the contract have no way of being retrieved by the BTAF team.

Tokens can be earned by the contract via staking fees or given to the contract to fund rewards. However, after the vesting period and the contract is no longer in use, the contract owners have no way of retrieving tokens owned by the contract.

Resolution: Add a onlyOwner withdraw feature that only allows excess leftover tokens to be withdrawn.

Status: Acknowledged

Medium

- (1) **Staking contracts:** Ever-growing array size with no control or disposal.



```
Staking.sol  
  
48     mapping (address => Stake[]) internal stakes;  
49     address[] investors;
```

After deployment, the investors and Stakes array will start to be filled up, with no method of disposing of addresses. This introduces the theoretical potential for a denial

of service once the array is filled. Moreover, the transaction to add investors or stakes would fail long before the array is filled up, because eventually the transaction would exceed the block gas limit when retrieving it.

By calling individual indexes, you have successfully mitigated the risk, stopping it from being a high level vulnerability, however Hashlock always advises to have control and disposal in arrays of this nature.

Resolution:

Removing addresses and Stakes in the claimReward and withdrawStake functions drastically reduces this risk.

Acknowledged: New function to limit investors and extreme quantities needed to cause problems will mitigate this issue.

(2) **Staking contracts:** Human input - unreliable deployment method and inability to audit staking specs.

Hashlock was sent 4 staking contracts;

- 90 Days: 65% annual yield (total 16.027% of stake given as reward at maturation):
<https://testnet.bscscan.com/address/0xBD08837459F554A9705547cE5E9b1Ad9137cE26a#code>
- 2 years: 25% annual yield (total 50% of stake given as reward at maturation):
<https://testnet.bscscan.com/address/0x9b1F2f136694813c47Db76d1fCc762b090aC3dAC#code>
- 3 years: 40% annual yield (total 120% of stake given as reward at maturation):
<https://testnet.bscscan.com/address/0x74Cda84486232aef701e3656D12a3fC42E7064ef#code>
- 1 day, 10% reward version for ease of testing:
<https://testnet.bscscan.com/address/0xCbF132A8Bc6bF64C22CC3004DD7d20bbEb127dA7#code>

All have the standard 20% initial burn rate, decreasing linearly.

The issue is that these specifications are not built into the contracts, each contract is currently the exact same (checked as each has the same MD5 Hash), and the variables need to be set manually in the constructor when the contracts are deployed, and they are immutable and can not be changed or amended after.

What this means is that if a typo or error occurs when the specs are manually typed in before deployment, they can't be changed. The contracts would need to be re-deployed. It also means that Hashlock was not able to check the staking variables at this stage of the audit.

Resolution: Set the staking variables in the state variable storage or have them pre-set in the constructor, so there is no human input when deploying the contracts.

Resolved: Staking parameters now hard coded into each contract.

Low

(1) **Staking.sol contracts:** Critical operation lacks event log.

Missing event log for:

- depositStake()

Resolution: Please write an event log for listed events.

Resolved

(2) **Staking.sol contracts:** Function input parameters lack of check.

Some functions require validation before execution.

Functions are:

- depositStake()

```

93  function depositStake(uint256 amount) public {
94      // Transfer the amount from the staker to the contract
95      IERC20(token).transferFrom(address(msg.sender), address(this), amount);

```

Resolution: We suggest using validation like for numerical variables that should be greater than 0 and for address type check variables that are not address(0). For percentage type variables, values should have some range like minimum 0 and maximum 100.

Resolved

(3) **Staking.sol contracts:** Function input parameters lack of check and gas inefficiency.

```

128  function claimReward(uint256 index) public {
129      // Get the stake
130      uint256 rewardAmount = previewInvestorClaimReward(address(msg.sender), index);
131
132      if (rewardAmount > 0) {
133          Stake storage stake = stakes[address(msg.sender)][index];
134          stake.totalRewardClaimed += rewardAmount;
135          IERC20(token).transfer(stake.staker, rewardAmount);
136          emit RewardClaimed(stake.staker, index, rewardAmount);
137      }
138  }

```

The function should only execute if RewardAmount > 0. When using an if statement, the function still executes, using gas, but no interactions get triggered.

By using a require statement instead after rewardAmount is calculated, we can roll back the function call, revert and refund gas. It is slightly more efficient and suitable here.

Example new function:

```
reccomendation.sol

1  function claimReward(uint256 index) public {
2  // @dev Get the stake
3  uint256 rewardAmount = previewInvestorClaimReward(address(msg.sender), index);
4
5
6  // @dev revert if no reward
7  require(rewardAmount != 0, "No claimable reward");
8
9
10 // @dev execute transaction
11     Stake storage stake = stakes[address(msg.sender)][index];
12     stake.totalRewardClaimed += rewardAmount;
13     IERC20(token).transfer(stake.staker, rewardAmount);
14
15
16 // @dev emit eventlog
17     emit RewardClaimed(stake.staker, index, rewardAmount);
18
19 }
20
21 }
```

Resolved

(4) **Staking.sol contracts:** depositStake function ordering.

The depositStake function transfers user tokens in its first line, and then proceeds to update the arrays and mappings.

The ideal way to order transacting functions is to make checks first, then implement effects by updating balances and state variables, and then make external interactions last.

```
Staking.sol

93 function depositStake(uint256 amount) public {
94     // Transfer the amount from the staker to the contract
95     IERC20(token).transferFrom(address(msg.sender), address(this), amount);
```

Resolved

(5) **Staking.sol contracts:** Unused inheritance.

The staking contracts inherit `onlyOwner`, but do not have any `onlyOwner` functions. This is a waste of gas and makes the contract misleading.

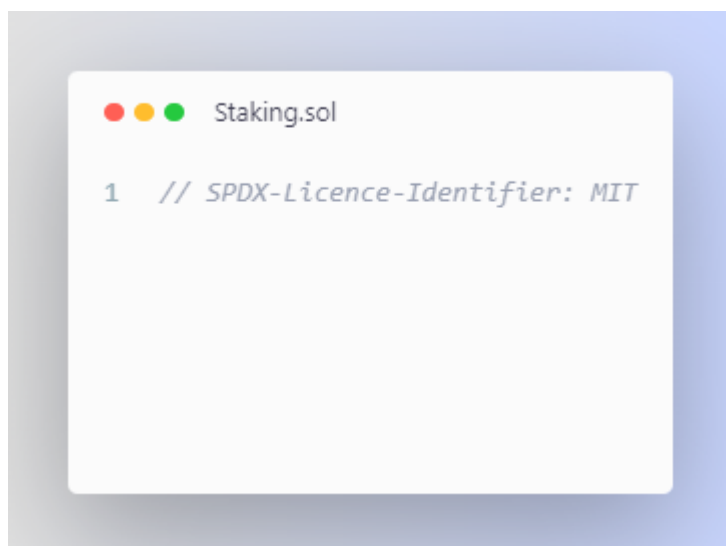
```
Staking.sol

15 contract Staking is Ownable {
```

Resolved

Very Low / Informational / Best practices:

(1) **Staking.sol contracts:** Spelling mistake.



Spelling mistake in SPDX licence identifier.

`// SPDX-Licence-Identifier: MIT`

"licence" should be **"license"**, to be consistent with the utilised US spelling.

Resolution: Correct the spelling.

Resolved

(2) **BTAF Protocol:** Solidity version not consistent and not the most recent version.

OpenZeppelin contracts are ^0.8.0, whilst custom contracts are ^0.8.3. The most recent solidity update is 0.8.17.

Resolution: We suggest deploying all contracts with ^0.8.17.

Resolved

(2) **Staking contracts:** Many functions lack comments before or within their logic.

```
Staking.sol

64
65     function getStake(uint256 index) public view returns (Stake memory) {
66         Stake memory stake = stakes[address(msg.sender)][index];
67         require(stake.startDate > 0, "Stake does not exist");
68         return stake;
69     }
70
71     function getInvestorStake(address investor, uint256 index) public view returns (Stake memory) {
72         Stake memory stake = stakes[investor][index];
73         require(stake.startDate > 0, "Stake does not exist");
74         return stake;
75     }
76
77     function getStakeCount() public view returns (uint256) {
78         return stakes[address(msg.sender)].length;
79     }
80
81     function getInvestorStakeCount(address investor) public view returns (uint256) {
82         return stakes[investor].length;
83     }
84
85     function getInvestorCount() public view returns (uint256) {
86         return investors.length;
87     }
88
89     function getInvestor(uint256 index) public view returns (address) {
90         return investors[index];
91     }
```

Most functions are self explanatory, but more comments would increase ease of reading and clarity.

Resolution: Add comments to every function.

Resolved

Centralisation

The BTAF protocol values security and utility over decentralisation. However, the staking contract features no onlyOwner functions, and the invoiceHandler contract only features the ability to set invoice signer, and the ability to sign them. Therefore, the contracts in this audit are not overly centralised.

The owner executable functions within the protocol increase security and functionality but depend highly on private key security, and internal team responsibility.



Centralised

Decentralised

Owner executable functions

InvoiceHandler.sol

- setInvoiceSigner: Owner can change the registered invoice signer for the contract.

Conclusion

After Hashlocks analysis, these BTAF contracts seem to have a sound and well tested code base, pending the resolution of our findings. Overall, most of the code is correctly ordered and follows industry best practices.

Since possible test cases can be unlimited, we provide no such guarantee of future outcomes. We have used both static tools and manual line-by-line analysis to minimise the probability of contract failure.

Pending the resolution of our findings, Hashlock believes BTAF's solidity code base is ready for deployment.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

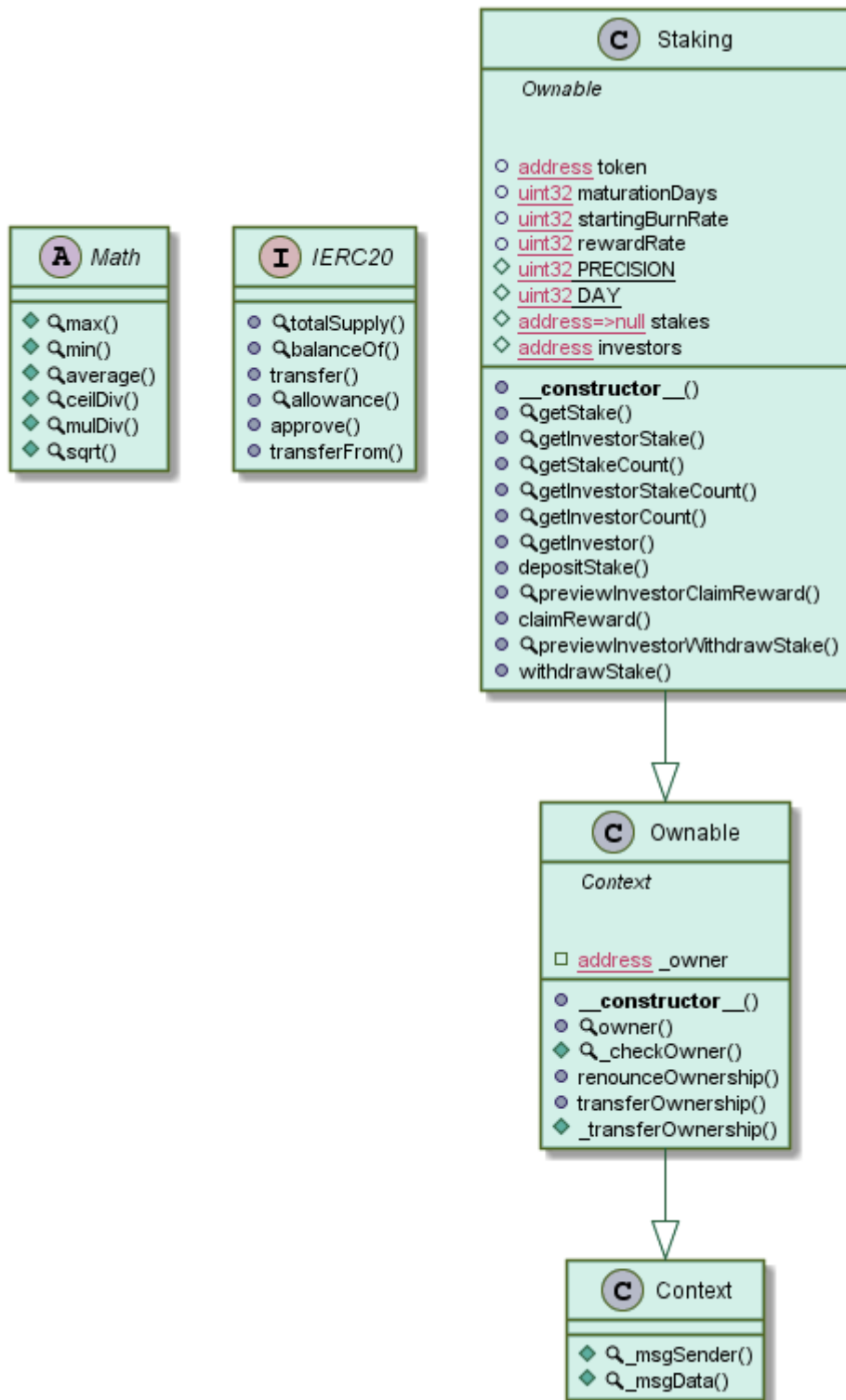
Contact: info@hashlock.com.au



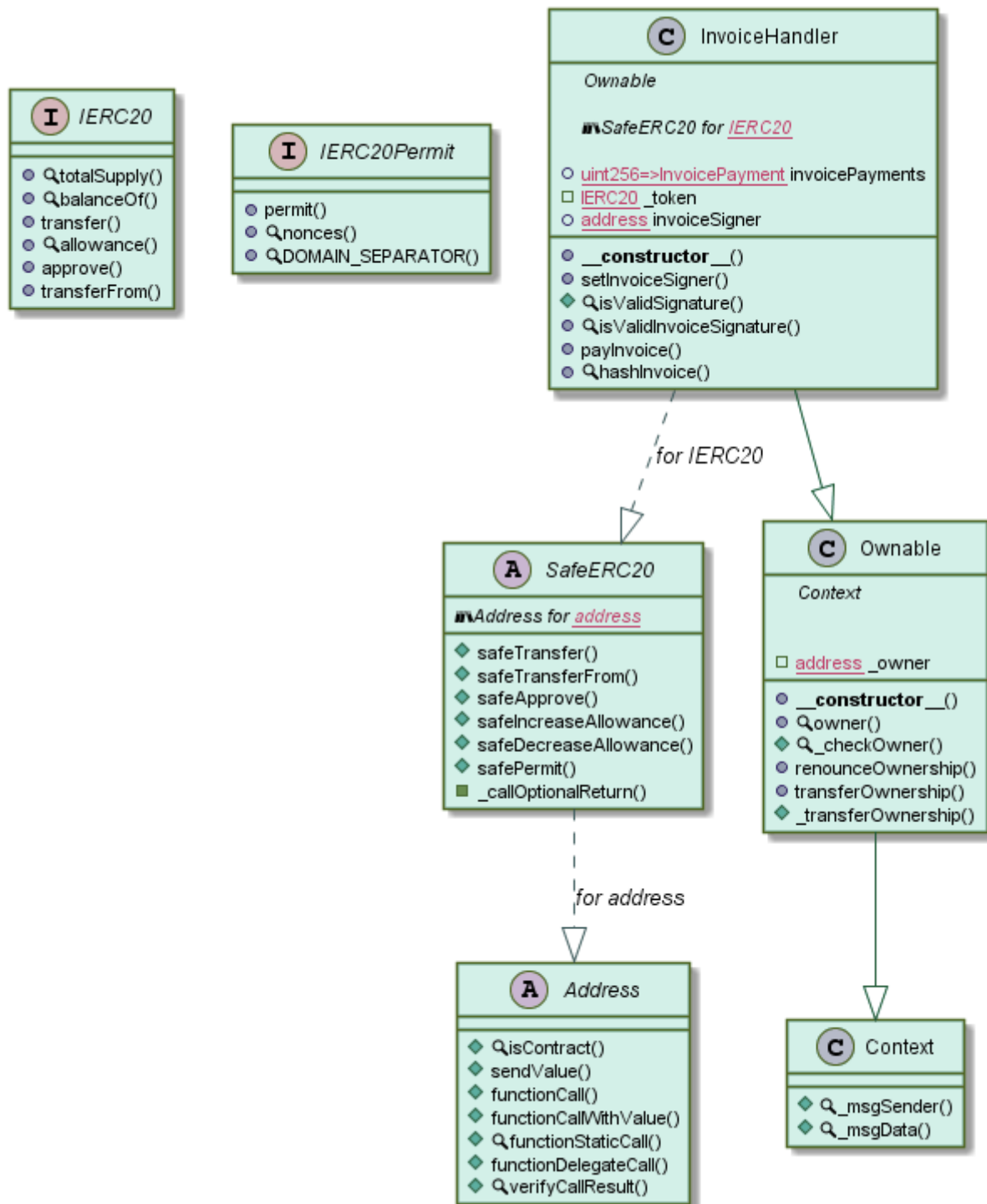
Appendix

Code Flow Diagram - BTAF Protocol

Staking Diagram



InvoiceHandler Diagram



Slither Results Log

Slither Log >> Staking.sol

```

Staking.constructor(address,uint32,uint32,uint32).token_ (Staking.sol#414) lacks a zero-check on :
- token = token_ (Staking.sol#415)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation

Reentrancy in Staking.depositStake(uint256) (Staking.sol#449-461):
  External calls:
  - IERC20(token).transferFrom(address(msg.sender),address(this),amount) (Staking.sol#451)
  State variables written after the call(s):
  - investors.push(address(msg.sender)) (Staking.sol#456)
  - stakes[address(msg.sender)].push(stake) (Staking.sol#459)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-2

Reentrancy in Staking.claimReward(uint256) (Staking.sol#484-494):
  External calls:
  - IERC20(token).transfer(stake.staker,rewardAmount) (Staking.sol#491)
  Event emitted after the call(s):
  - RewardClaimed(stake.staker,index,rewardAmount) (Staking.sol#492)
Reentrancy in Staking.withdrawStake(uint256) (Staking.sol#516-532):
  External calls:
  - IERC20(token).transfer(address(1),burnAmount) (Staking.sol#524)
  - IERC20(token).transfer(stake.staker,withdrawAmount) (Staking.sol#529)
  Event emitted after the call(s):
  - StakeWithdrawn(stake.staker,index,withdrawAmount,burnAmount) (Staking.sol#531)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3

Staking.getStake(uint256) (Staking.sol#421-425) uses timestamp for comparisons
  Dangerous comparisons:
  - require(bool,string)(stake.startDate > 0,Stake does not exist) (Staking.sol#423)
Staking.getInvestorStake(address,uint256) (Staking.sol#427-431) uses timestamp for comparisons
  Dangerous comparisons:
  - require(bool,string)(stake.startDate > 0,Stake does not exist) (Staking.sol#429)
Staking.depositStake(uint256) (Staking.sol#449-461) uses timestamp for comparisons
  Dangerous comparisons:
  - stakes[address(msg.sender)].length == 0 (Staking.sol#455)
Staking.previewInvestorClaimReward(address,uint256) (Staking.sol#463-481) uses timestamp for comparisons
  Dangerous comparisons:
  - stakes[address(msg.sender)].length == 0 (Staking.sol#455)
Staking.previewInvestorClaimReward(address,uint256) (Staking.sol#463-481) uses timestamp for comparisons
  Dangerous comparisons:
  - stake.maturationDate > block.timestamp (Staking.sol#469)
Staking.claimReward(uint256) (Staking.sol#484-494) uses timestamp for comparisons
  Dangerous comparisons:
  - rewardAmount > 0 (Staking.sol#488)
Staking.previewInvestorWithdrawStake(address,uint256) (Staking.sol#496-514) uses timestamp for comparisons
  Dangerous comparisons:
  - require(bool,string)(stake.dateWithdrawn == 0,Stake has already been withdrawn) (Staking.sol#501)
  - stake.maturationDate > block.timestamp (Staking.sol#505)
Staking.withdrawStake(uint256) (Staking.sol#516-532) uses timestamp for comparisons
  Dangerous comparisons:
  - burnAmount > 0 (Staking.sol#522)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp

Math.mulDiv(uint256,uint256,uint256) (Staking.sol#49-129) uses assembly
  - INLINE ASM (Staking.sol#60-64)
  - INLINE ASM (Staking.sol#80-87)
  - INLINE ASM (Staking.sol#94-103)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage

Context._msgData() (Staking.sol#301-303) is never used and should be removed
Math.average(uint256,uint256) (Staking.sol#28-31) is never used and should be removed
Math.ceilDiv(uint256,uint256) (Staking.sol#39-42) is never used and should be removed
Math.max(uint256,uint256) (Staking.sol#13-15) is never used and should be removed
Math.min(uint256,uint256) (Staking.sol#20-22) is never used and should be removed
Math.sqrt(uint256) (Staking.sol#152-208) is never used and should be removed
Math.sqrt(uint256,Math.Rounding) (Staking.sol#213-219) is never used and should be removed
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dead-code

Pragma version^0.8.4 (Staking.sol#2) allows old versions
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
Staking.sol analyzed (5 contracts with 84 detectors), 36 result(s) found

```

Slither Log >> InvoiceHandler.sol

```

Reentrancy in InvoiceHandler.payInvoice(uint256,uint256,InvoiceHandler.Signature) (InvoiceHandler.sol#609-628):
  External calls:
    - _token.safeTransferFrom(msg.sender,owner(),amount) (InvoiceHandler.sol#620)
  Event emitted after the call(s):
    - InvoicePaid(invoiceId,msg.sender,amount) (InvoiceHandler.sol#627)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3

Address.verifyCallResult(bool,bytes,string) (InvoiceHandler.sol#320-340) uses assembly
  - INLINE ASM (InvoiceHandler.sol#332-335)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#assembly-usage

Address.functionCall(address,bytes) (InvoiceHandler.sol#204-206) is never used and should be removed
Address.functionCallWithValue(address,bytes,uint256) (InvoiceHandler.sol#233-239) is never used and should be removed
Address.functionDelegateCall(address,bytes) (InvoiceHandler.sol#293-295) is never used and should be removed
Address.functionDelegateCall(address,bytes,string) (InvoiceHandler.sol#303-312) is never used and should be removed
Address.functionStaticCall(address,bytes) (InvoiceHandler.sol#266-268) is never used and should be removed
Address.functionStaticCall(address,bytes,string) (InvoiceHandler.sol#276-285) is never used and should be removed
Address.sendValue(address,uint256) (InvoiceHandler.sol#179-184) is never used and should be removed
Context.msgData() (InvoiceHandler.sol#446-448) is never used and should be removed
SafeERC20.safeApprove(IERC20,address,uint256) (InvoiceHandler.sol#369-382) is never used and should be removed
SafeERC20.safeDecreaseAllowance(IERC20,address,uint256) (InvoiceHandler.sol#393-404) is never used and should be removed
SafeERC20.safeIncreaseAllowance(IERC20,address,uint256) (InvoiceHandler.sol#384-391) is never used and should be removed
SafeERC20.safePermit(IERC20Permit,address,address,uint256,uint256,uint8,bytes32,bytes32) (InvoiceHandler.sol#406-420) is never
used and should be removed
SafeERC20.safeTransfer(IERC20,address,uint256) (InvoiceHandler.sol#345-351) is never used and should be removed
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dead-code

Pragma version^0.8.4 (InvoiceHandler.sol#2) allows old versions
solc-0.8.4 is not recommended for deployment
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity

Low level call in Address.sendValue(address,uint256) (InvoiceHandler.sol#179-184):
  - (success) = recipient.call{value: amount}() (InvoiceHandler.sol#182)
Low level call in Address.functionCallWithValue(address,bytes,uint256,string) (InvoiceHandler.sol#247-258):
  - (success,returndata) = target.call{value: value}(data) (InvoiceHandler.sol#256)
Low level call in Address.functionStaticCall(address,bytes,string) (InvoiceHandler.sol#276-285):
  - (success,returndata) = target.staticcall(data) (InvoiceHandler.sol#283)
Low level call in Address.functionDelegateCall(address,bytes,string) (InvoiceHandler.sol#303-312):
  - (success,returndata) = target.delegatecall(data) (InvoiceHandler.sol#310)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls

Function IERC20Permit.DOMAIN_SEPARATOR() (InvoiceHandler.sol#125) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions

InvoiceHandler._token (InvoiceHandler.sol#535) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immutable
InvoiceHandler.sol analyzed (7 contracts with 84 detectors), 24 result(s) found

```

Solidity Static Analysis

Staking.sol

Security

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in `Staking.withdrawStake(uint256)`: Could potentially lead to re-entrancy vulnerability. Note: Modifiers are currently not considered by this static analysis.
[more](#)

Pos: 516:6:

Block timestamp:

Use of "block.timestamp": "block.timestamp" can be influenced by miners to a certain degree. That means that a miner can "choose" the block.timestamp, to a certain degree, to change the outcome of a transaction in the mined block.

[more](#)

Pos: 526:32:

Gas & Economy

Gas costs:

Gas requirement of function `Staking.maturationDays` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 373:6:

Gas costs:

Gas requirement of function `Staking.withdrawStake` is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 516:6:

Miscellaneous

Constant/View/Pure functions:

Staking.previewInvestorWithdrawStake(address,uint256) : Is constant but potentially should not be. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 496:6:

Similar variable names:

Staking.withdrawStake(uint256) : Variables have very similar names "stake" and "stakes". Note: Modifiers are currently not considered by this static analysis.

Pos: 531:30:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 501:10:

Data truncated:

Division of integer values yields an integer value again. That means e.g. $10 / 100 = 0$ instead of 0.1 since the result is an integer again. This does not hold for division of (only) literal values since those yield rational constants.

Pos: 206:33:

Security

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in `SafeERC20.safeDecreaseAllowance(contract IERC20,address,uint256)`: Could potentially lead to re-entrancy vulnerability. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 393:4:

Inline assembly:

The Contract uses inline assembly, this is only advised in rare cases. Additionally static analysis modules do not parse inline Assembly, this can lead to wrong analysis results.

[more](#)

Pos: 332:16:

Low level calls:

Use of "delegatecall": should be avoided whenever possible. External code, that is called can change the state of the calling contract and send ether from the caller's balance. If this is wanted behaviour, use the Solidity library feature if possible.

[more](#)

Pos: 310:50:

Gas & Economy

Gas costs:

Gas requirement of function InvoiceHandler.payInvoice is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 609:4:

Gas costs:

Gas requirement of function InvoiceHandler.hashInvoice is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)

Pos: 635:4:

Miscellaneous

Constant/View/Pure functions:

InvoiceHandler.hashInvoice(uint256,uint256) : Is constant but potentially should not be. Note: Modifiers are currently not considered by this static analysis.

[more](#)

Pos: 635:4:

No return:

IERC20Permit.DOMAIN_SEPARATOR(): Defines a return type but never explicitly returns a value.

Pos: 125:4:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 618:8:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.

[more](#)

Pos: 619:8:

Solhint Linter

Staking.sol

```
Staking.sol:54:18: Error: Parse error: missing ';' at '{'  
Staking.sol:198:18: Error: Parse error: missing ';' at '{'
```

InvoiceHandler.sol

```
InvoiceHandler.sol:398:18: Error: Parse error: missing ';' at '{'
```

Software analysis result:

Our Automated software tools reported many false positive results and some informational issues. However, these issues can be safely ignored.

#Hashlock.

